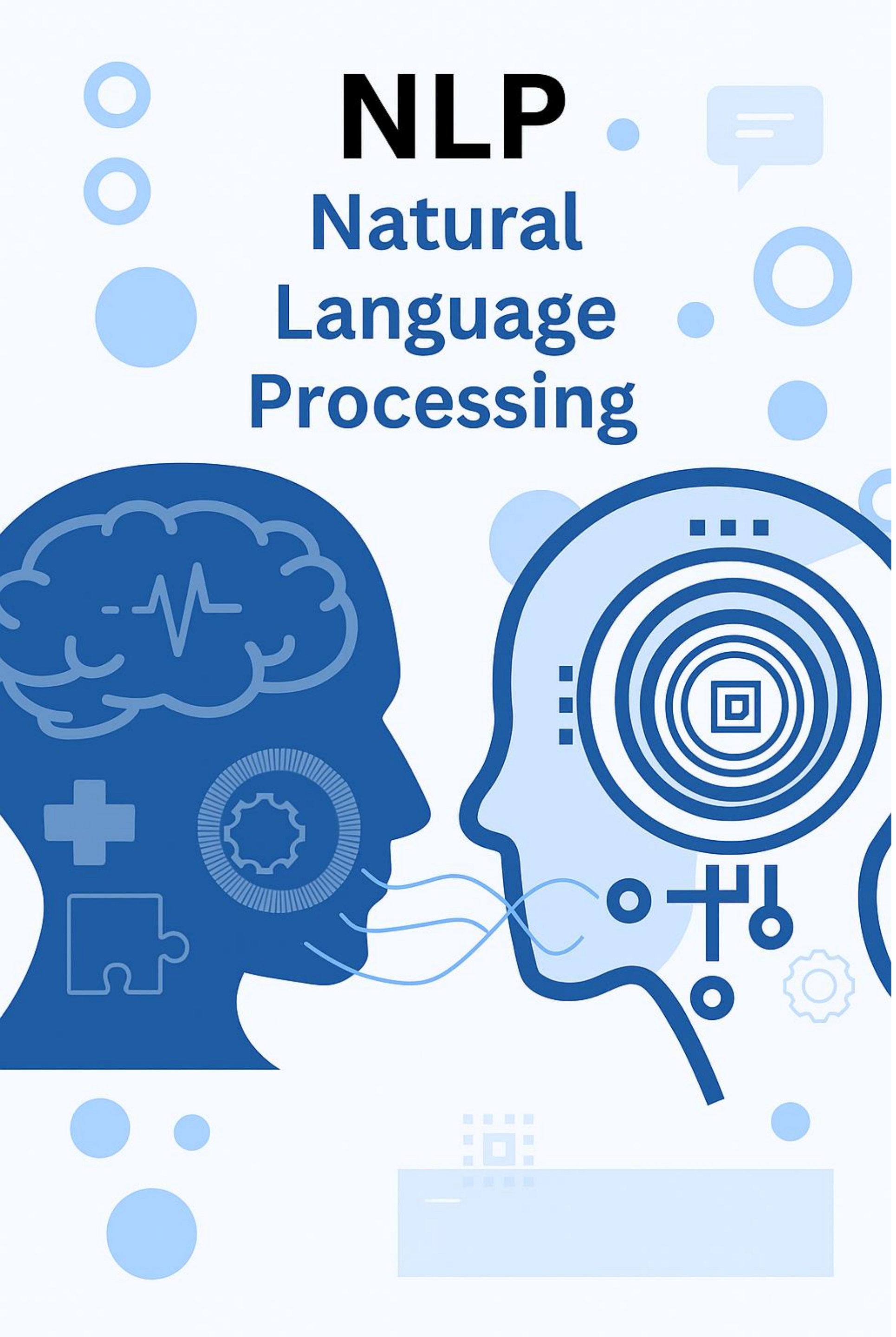


NATURAL LANGUAGE PROCESSING (NLP)

PMDS606L

MODULE 4

LECTURE 7

Dr. Kamanasish Bhattacharjee

Assistant Professor

Dept. of Analytics, SCOPE, VIT

Finite State Transducer (FST)

- In NLP, a Finite State Transducer (FST) is a computational model that transforms one sequence of symbols into another, mapping input strings to output strings. Unlike a standard Finite State Automaton (FSA) which recognizes strings, an FST generates a corresponding output for each input, serving as a "smart helper" for tasks like morphological analysis (e.g., changing "cat" to "cat-plural"), spell checking, and machine translation.
- Mapping a word to its morphological analysis (e.g., “running” → “run+ing”).
- Translating between languages (e.g., “cat” → “gato” in Spanish).

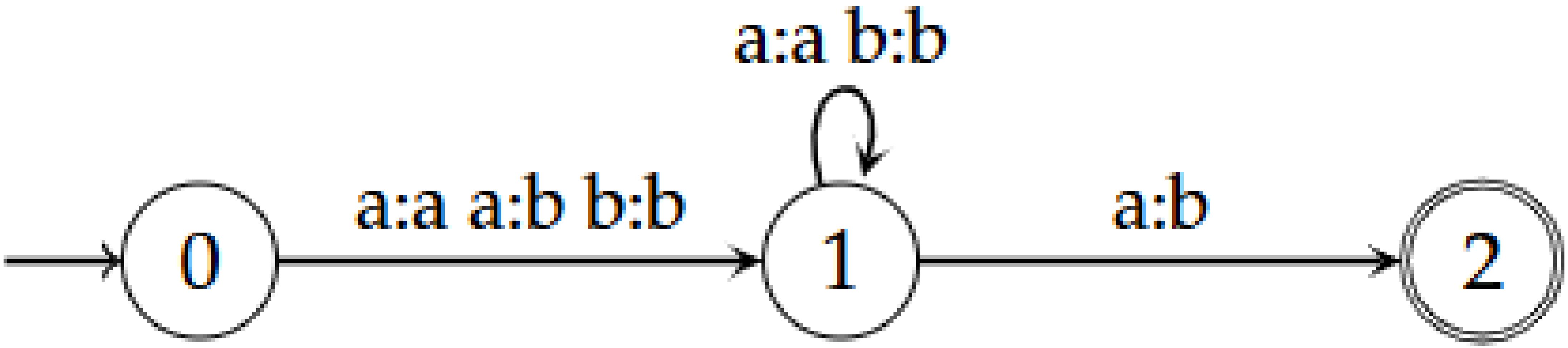
Mathematical Representation: FST

The transition function $\delta : Q \times (\Sigma \cup \varepsilon) \rightarrow Q \times (\Delta \cup \varepsilon)$ defines the transitions of the FST, where ε represents the empty string. Formally, an FST can be represented as a 5-tuple $T = (Q, \sum, \Delta, \delta, F)$ where:

- Q is a finite set of states.
- $\sum$ is a finite input alphabet.
- Δ is a finite output alphabet.
- δ is the transition function.
- $F \subseteq Q$ is a set of final states.

The transition function δ is typically defined as a mapping from a state and an input symbol to a new state and an output symbol.

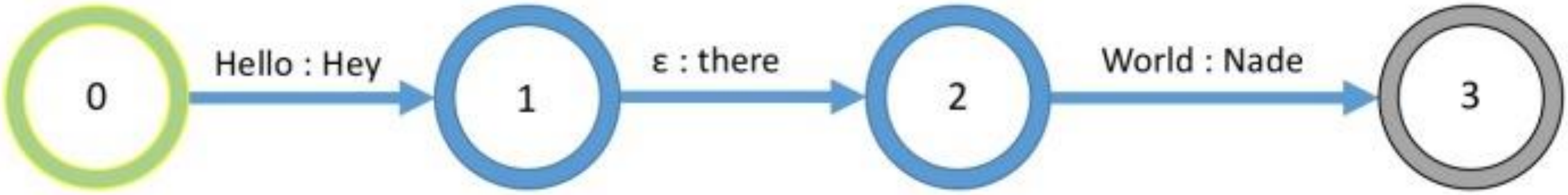
For a transition $\delta(q, a) = (p, b)$, it means that when the FST is in state q and reads input symbol a, it transitions to state p while producing output symbol b.

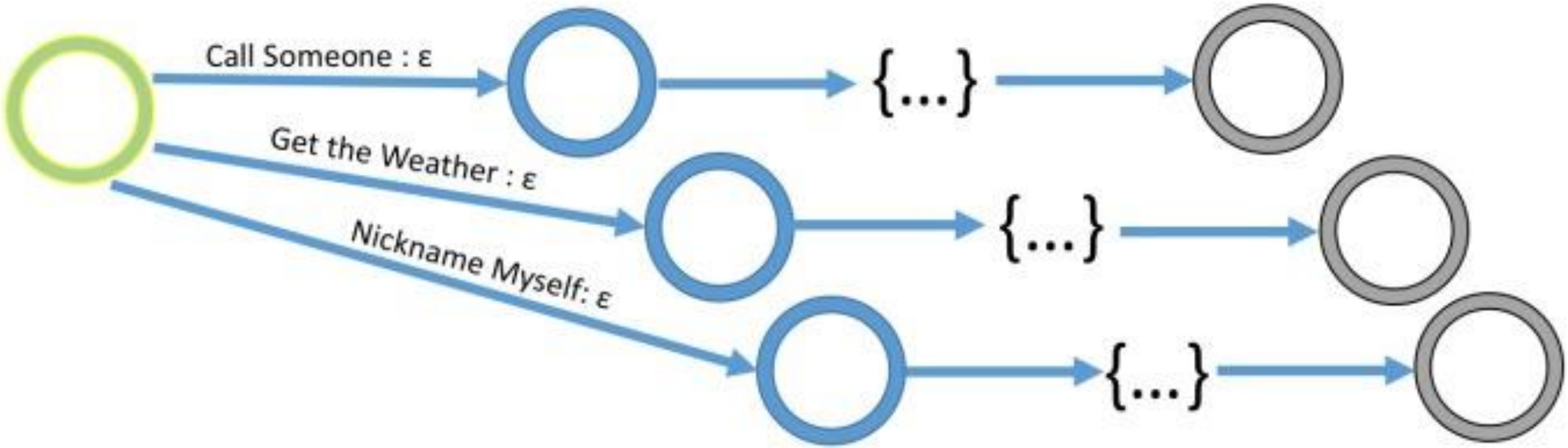


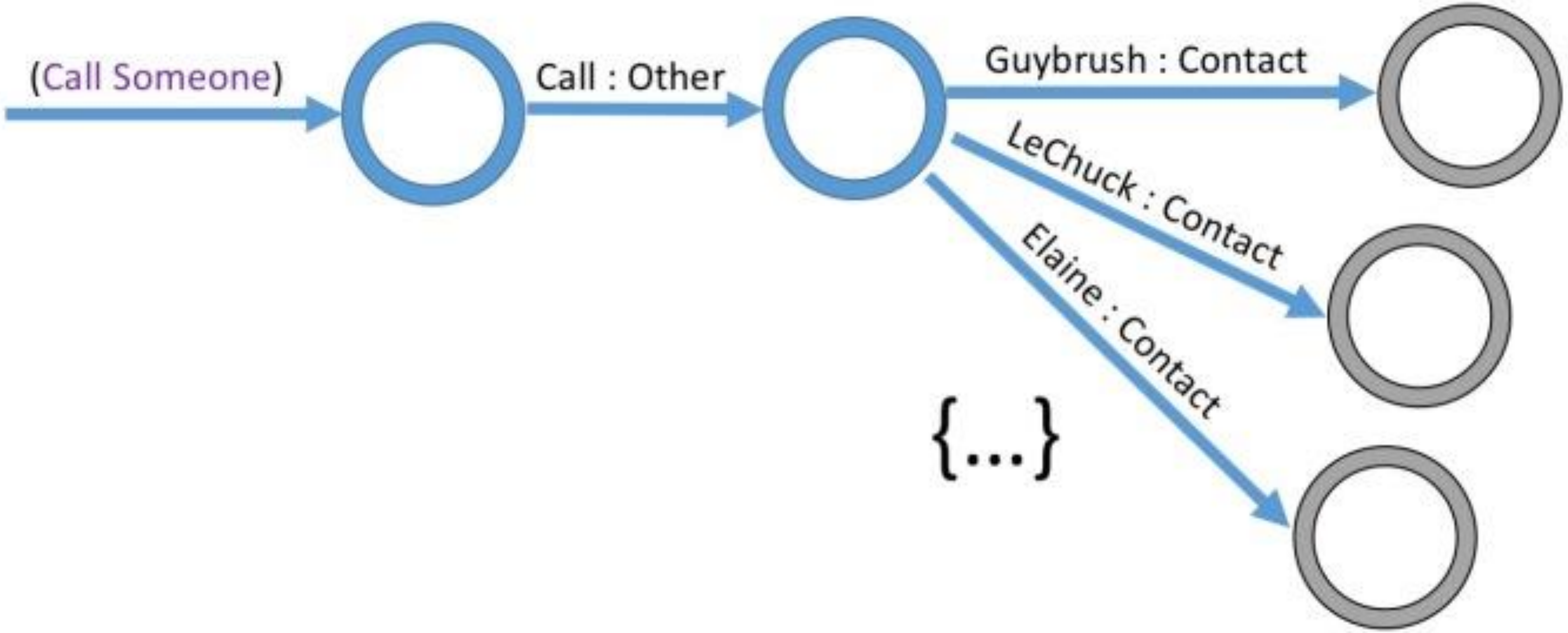
$$\text{babba} \longrightarrow \text{babbb}$$

$$\text{aba} \longrightarrow \text{bbb}$$

$$\text{aba} \longrightarrow \text{abb}$$

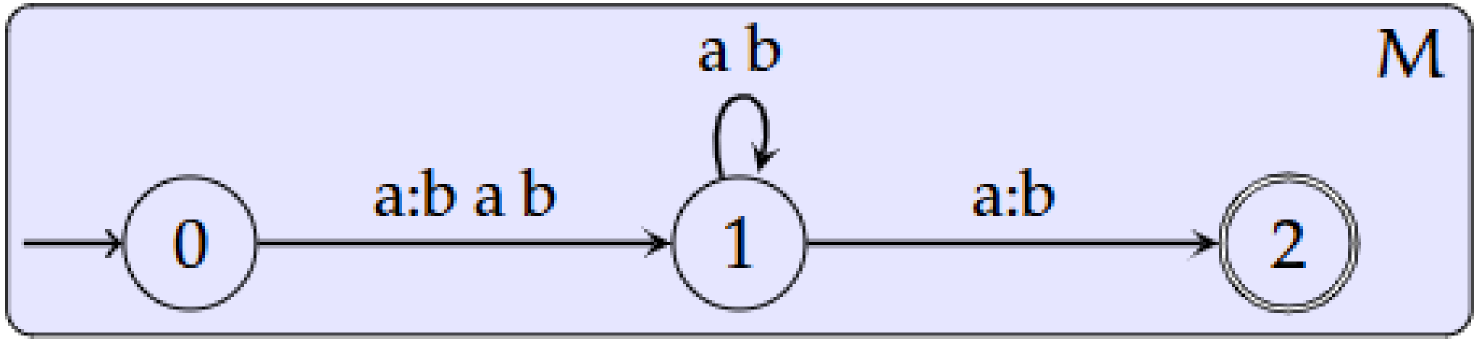

0
Hello : Hey
1
ε : there
2
World : Nade
3

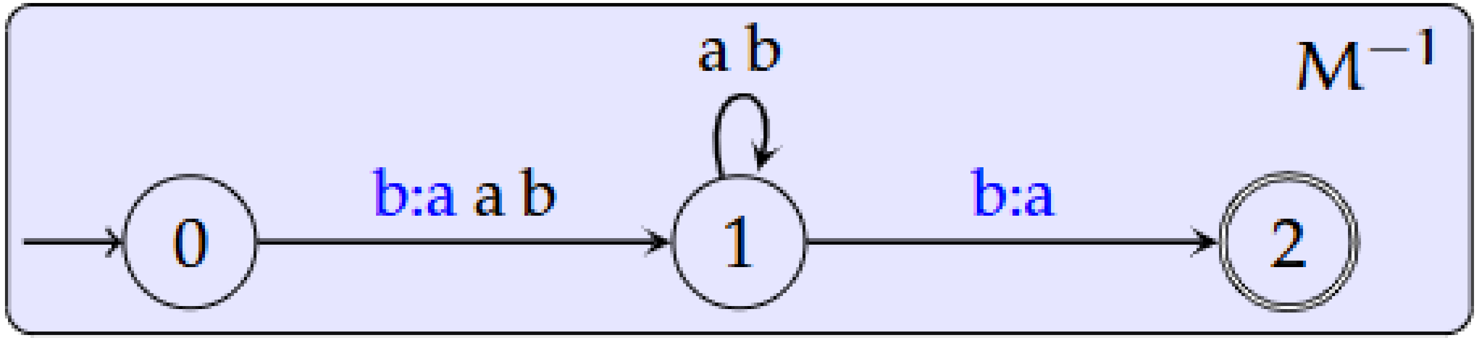

Call Someone : ε
{...}
Get the Weather : ε
{...}
Nickname Myself: ε
{...}

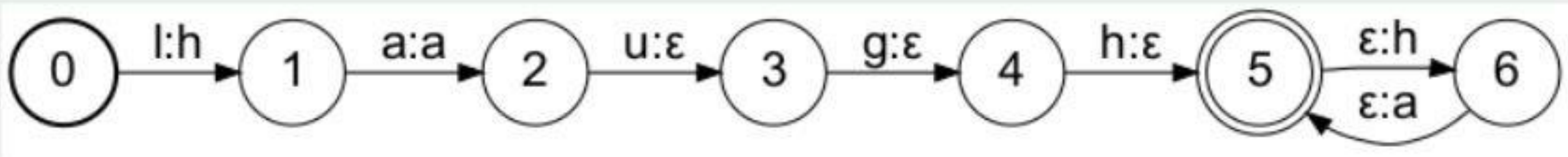

(Call Someone)
Call : Other
Guybrush : Contact
LeChuck : Contact
Elaine : Contact
{...}

FST inversion

- Since FST encodes a relation, it can be reversed
- Inverse of an FST swaps the input symbols with output symbols
- We indicate inverse of an FST M with M^{-1}







Laughing machine T_{laugh}

The input string laugh is mapped to the infinite set $\{ha^n | n \geq 1\}$

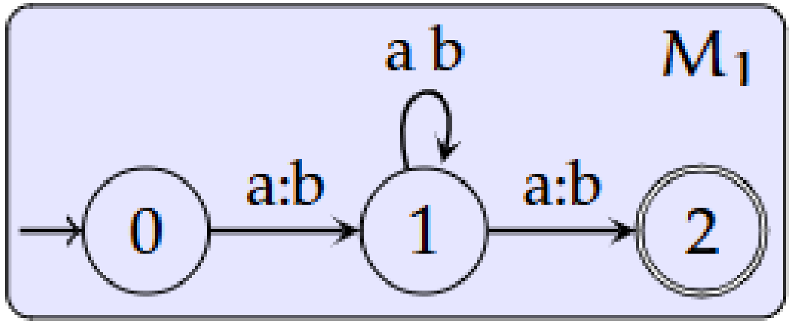
(laugh, ha)

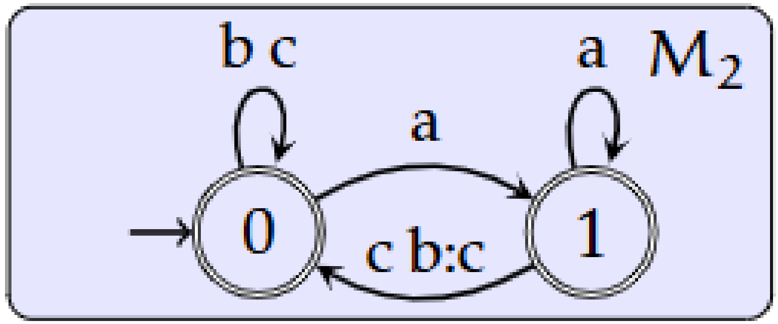
(laugh, haa)

(laugh, haaa)

FST composition

sequential application





$$\mathrm{aa} \xrightarrow{M_1}$$

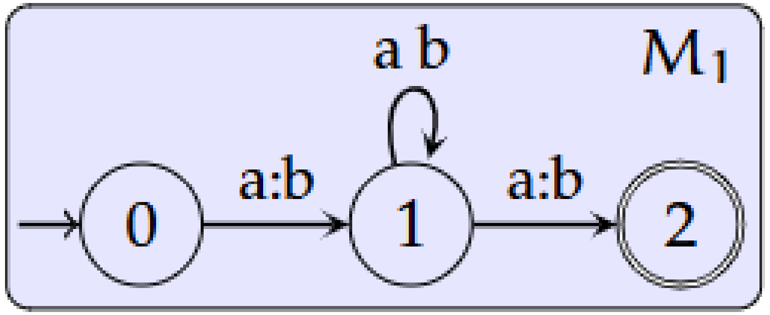
$$\mathrm{bb} \xrightarrow{M_1}$$

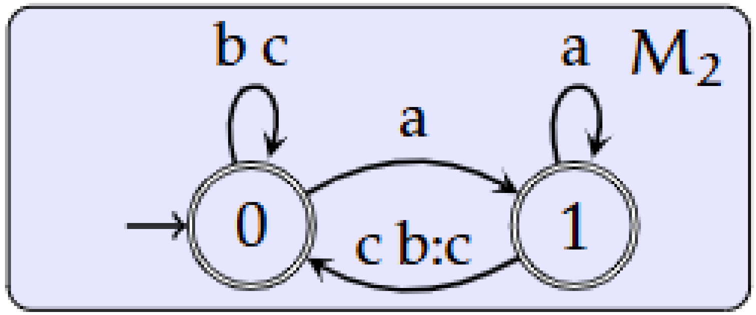
$$\mathrm{aaaa} \xrightarrow{M_1}$$

$$\mathrm{abaa} \xrightarrow{M_1}$$

FST composition

sequential application

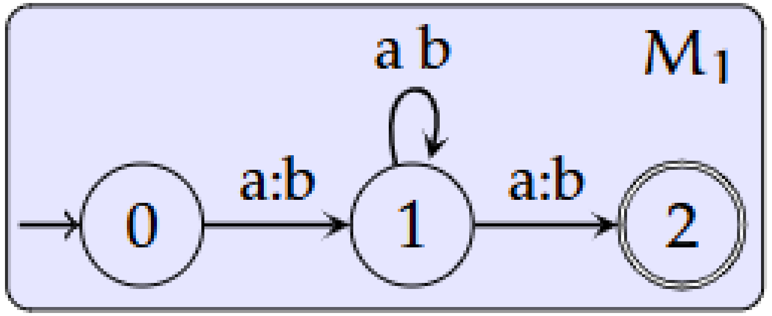


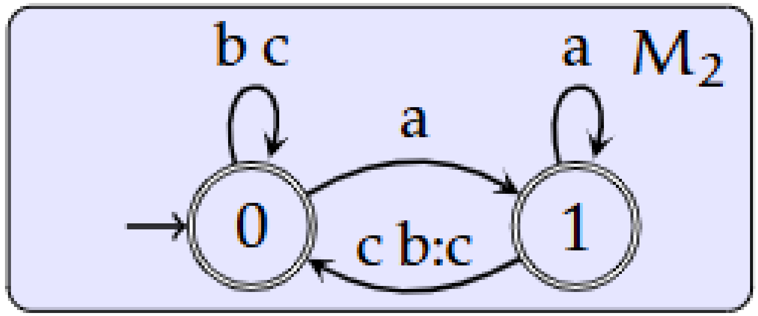


$$
\begin{array}{llllll}
\mathrm{aa} & \xrightarrow{M_1} & \mathrm{bb} & \xrightarrow{M_2} \\
\mathrm{bb} & \xrightarrow{M_1} & \varnothing & \xrightarrow{M_2} \\
\mathrm{aaaa} & \xrightarrow{M_1} & \mathrm{baab} & \xrightarrow{M_2} \\
\mathrm{abaa} & \xrightarrow{M_1} & \mathrm{bbab} & \xrightarrow{M_2}
\end{array}
$$

FST composition

sequential application





$$\begin{array}{lcccc}
 & \multicolumn{3}{c}{\xrightarrow{\qquad\qquad M_1 \circ M_2 \qquad\qquad}} & \\
aa & \xrightarrow{M_1} & bb & \xrightarrow{M_2} & bb \\
bb & \xrightarrow{M_1} & \varnothing & \xrightarrow{M_2} & \varnothing \\
aaaa & \xrightarrow{M_1} & baab & \xrightarrow{M_2} & baac \\
abaa & \xrightarrow{M_1} & bbab & \xrightarrow{M_2} & bbac
\end{array}$$